

Lab-15.5

Course : AI Assisted Coding

Topic: Backend API Development: Creating RESTful services with AI

Name : Mohammed Sabir

Hall Ticket No : 2303A51506

Batch : 22

Task-01

Final Optimal Prompt:

Create a RESTful API for student attendance tracking using any backend framework (e.g., Flask, Node.js). Implement the following endpoints:

- GET /attendance → Retrieve all attendance records
- POST /attendance → Add a new attendance record
- PUT /attendance/{id} → Update an existing record
- DELETE /attendance/{id} → Delete a record

Ensure all requests and responses use JSON format and the API is well-structured and easy to understand.

Code Screenshot :

```

13  """
14  from flask import Flask, request, jsonify
15
16  app = Flask(__name__)
17
18  # In-memory database (list of records)
19  attendance_data = []
20  current_id = 1
21  # -----
22  # GET /attendance
23  # Retrieve all attendance records
24  # -----
25  @app.route('/attendance', methods=['GET'])
26  def get_attendance():
27      return jsonify(attendance_data), 200
28  # -----
29  # POST /attendance
30  # Add new attendance record
31  # -----
32  @app.route('/attendance', methods=['POST'])
33  def add_attendance():
34      global current_id
35
36      data = request.get_json()
37
38      # Validate input
39      if not data or 'name' not in data or 'status' not in data:
40          return jsonify({"error": "Name and status are required"}), 400
41
42      record = {
43          "id": current_id,
44          "name": data["name"],
45          "status": data["status"] # Present / Absent
46      }
47
48      attendance_data.append(record)
49      current_id += 1
50
51      return jsonify(record), 201
52  # -----
53  # PUT /attendance/<id>
54  # Update existing record
55  # -----
56  @app.route('/attendance/<int:id>', methods=['PUT'])
57  def update_attendance(id):
58      data = request.get_json()
59
60      for record in attendance_data:
61          if record["id"] == id:
62              record["name"] = data.get("name", record["name"])
63              record["status"] = data.get("status", record["status"])
64
65              return jsonify(record), 200
66
67      return jsonify({"error": "Record not found"}), 404
68  # -----
69  # DELETE /attendance/<id>
70  # Delete a record
71  # -----
72  @app.route('/attendance/<int:id>', methods=['DELETE'])
73  def delete_attendance(id):
74      global attendance_data
75
76      for record in attendance_data:
77          if record["id"] == id:
78              attendance_data = [r for r in attendance_data if r["id"] != id]
79              return jsonify({"message": "Record deleted"}), 200
80
81      return jsonify({"error": "Record not found"}), 404
82  # -----
83  # Run the server
84  # -----
85  if __name__ == '__main__':
86      app.run(debug=True)

```

Output Screenshot:

GET

Query Headers ² Auth **Body ¹** Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 [
2   {
3     "id": 1,
4     "name": "Sabin",
5     "status": "Present"
6   }
7 ]
```

Status: **200 OK** Size: **70 Bytes** Time: **13 ms**

Response Headers ⁵ Cookies Results Docs {} ≡

```
1 [
2   {
3     "id": 1,
4     "name": "Sabin",
5     "status": "Present"
6   }
7 ]
```

POST

Query Headers ² Auth **Body ¹** Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "name": "Sabin",
3   "status": "Present"
4 }
```

Status: **201 CREATED** Size: **56 Bytes** Time: **56 ms**

Response Headers ⁵ Cookies Results Docs {} ≡

```
1 {
2   "id": 1,
3   "name": "Sabin",
4   "status": "Present"
5 }
```

PUT

Query Headers ² Auth **Body ¹** Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "status": "Absent"
3 }
```

Status: **405 METHOD NOT ALLOWED** Size: **153 Bytes** Time: **9 ms**

Response Headers ⁶ Cookies Results Docs {} ≡

```
1 <!doctype html>
2 <html lang=en>
3 <title>405 Method Not Allowed</title>
4 <h1>Method Not Allowed</h1>
5 <p>The method is not allowed for the requested URL.</p>
```

DELETE

Query Headers ² Auth **Body** Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1
```

Status: **200 OK** Size: **34 Bytes** Time: **10 ms**

Response Headers ⁵ Cookies Results Docs {} ≡

```
1 {
2   "message": "Record deleted"
3 }
```

Explanation/Justification/Observation (100 words / 5 – 6 sentence) :

The given Flask application successfully implements a basic RESTful API for student attendance tracking using an in-memory list as a temporary database. It supports all CRUD operations: retrieving records (GET), adding new entries (POST), updating existing records (PUT), and deleting records (DELETE). Input validation is partially handled in the POST

method, ensuring required fields are present. The use of incremental IDs helps uniquely identify records. However, since data is stored in memory, it will be lost when the server restarts. Additionally, error handling could be improved for robustness. Overall, the code is clean, readable, and suitable for learning and small-scale testing purposes.

Task-02

Final Optimal Prompt:

Create a RESTful API for warehouse inventory management using any backend framework (e.g., Flask, Node.js). Implement the following endpoints:

- GET /inventory → Retrieve all items
- POST /inventory → Add a new item
- PUT /inventory/{id} → Update item stock quantity
- DELETE /inventory/{id} → Remove an item

Ensure all data is handled in JSON format and add validation to prevent invalid or negative stock values.

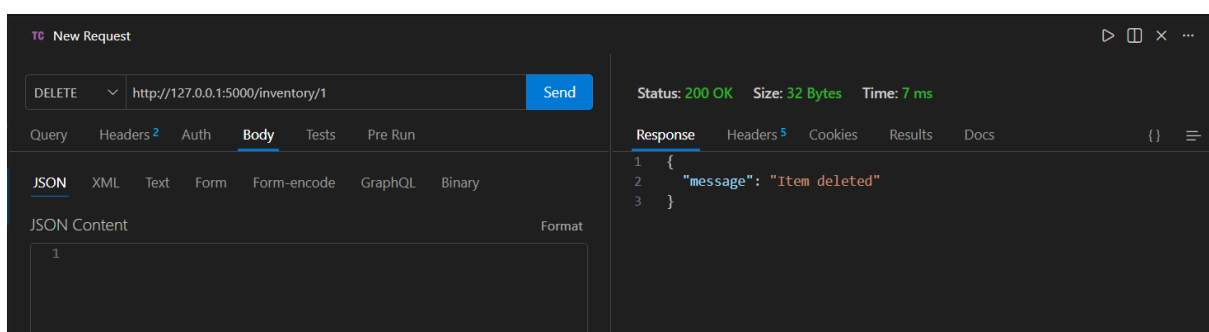
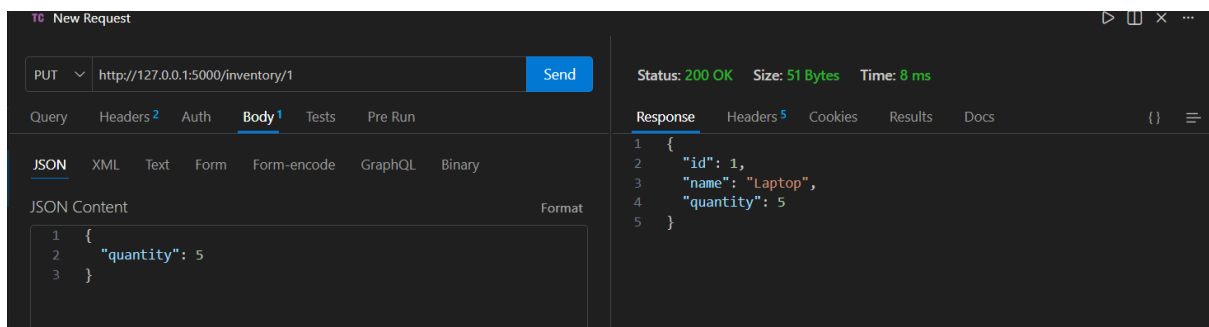
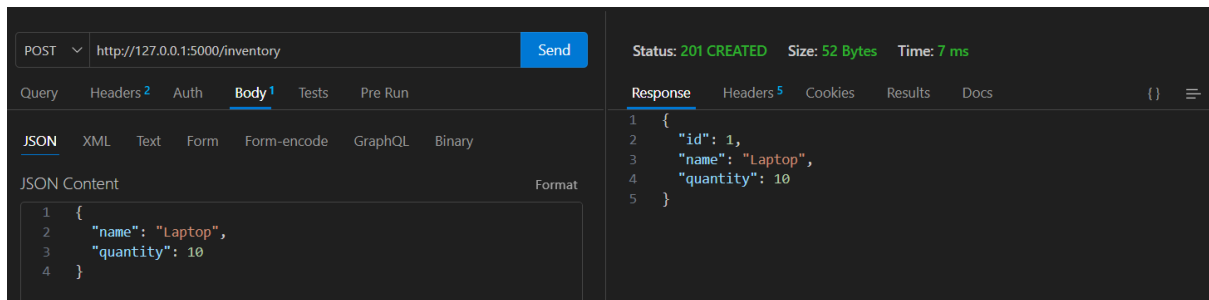
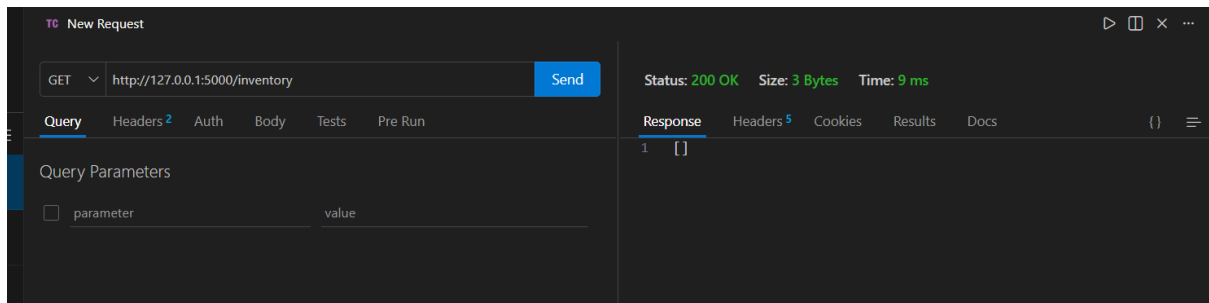
Code Screenshot :

```

98     """
99     from flask import Flask, request, jsonify
100     app = Flask(__name__)
101     # In-memory database
102     inventory = []
103     current_id = 1
104     # -----
105     # GET /inventory → Get all items
106     # -----
107     @app.route('/inventory', methods=['GET'])
108     def get_inventory():
109         return jsonify(inventory), 200
110     # -----
111     # POST /inventory → Add item
112     # -----
113     @app.route('/inventory', methods=['POST'])
114     def add_item():
115         global current_id
116         data = request.get_json()
117         # Validation
118         if not data or 'name' not in data or 'quantity' not in data:
119             return jsonify({"error": "Name and quantity required"}), 400
120         if not isinstance(data['quantity'], int) or data['quantity'] < 0:
121             return jsonify({"error": "Quantity must be a non-negative integer"}), 400
122         item = {
123             "id": current_id,
124             "name": data["name"],
125             "quantity": data["quantity"]
126         }
127         inventory.append(item)
128         current_id += 1
129         return jsonify(item), 201
130     # -----
131     # PUT /inventory/<id> → Update stock
132     # -----
133     @app.route('/inventory/<int:id>', methods=['PUT'])
134     def update_item(id):
135         data = request.get_json()
136         if 'quantity' not in data:
137             return jsonify({"error": "Quantity required"}), 400
138         if not isinstance(data['quantity'], int) or data['quantity'] < 0:
139             return jsonify({"error": "Quantity must be non-negative"}), 400
140         for item in inventory:
141             if item["id"] == id:
142                 item["quantity"] = data["quantity"]
143                 return jsonify(item), 200
144         return jsonify({"error": "Item not found"}), 404
145     # -----
146     # DELETE /inventory/<id> → Remove item
147     # -----
148     @app.route('/inventory/<int:id>', methods=['DELETE'])
149     def delete_item(id):
150         global inventory
151         for item in inventory:
152             if item["id"] == id:
153                 inventory = [i for i in inventory if i["id"] != id]
154                 return jsonify({"message": "Item deleted"}), 200
155         return jsonify({"error": "Item not found"}), 404
156     # -----
157     # Run server
158     # -----
159     if __name__ == '__main__':
160         app.run(debug=True)

```

Output Screenshot:



Explanation/Justification/Observation (100 words / 5 – 6 sentence) :

The Flask-based warehouse inventory API effectively demonstrates RESTful design principles by implementing CRUD operations for managing items. It uses an in-memory list to store inventory data, making it suitable for testing and small-scale applications. Input validation ensures that item names and quantities are provided, and prevents invalid or negative stock

values, improving data integrity. Each item is uniquely identified using an incremental ID, enabling precise updates and deletions. However, since the data is not persisted, all records are lost when the server restarts. Overall, the API is well-structured, easy to understand, and provides a strong foundation for real-world extensions with databases.

Task-03

Final Optimal Prompt:

Create a RESTful API for hotel room booking using any backend framework (e.g., Flask, Node.js). Implement the following endpoints:

- GET /rooms → Retrieve available rooms
- POST /rooms/book → Book a room
- GET /rooms/{id} → View booking details
- DELETE /rooms/{id} → Cancel a booking

Ensure all requests and responses use JSON format and implement proper error handling for unavailable or already booked rooms.

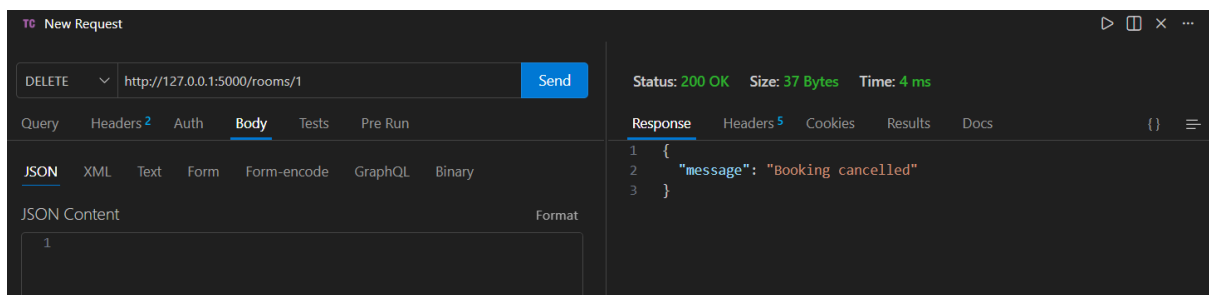
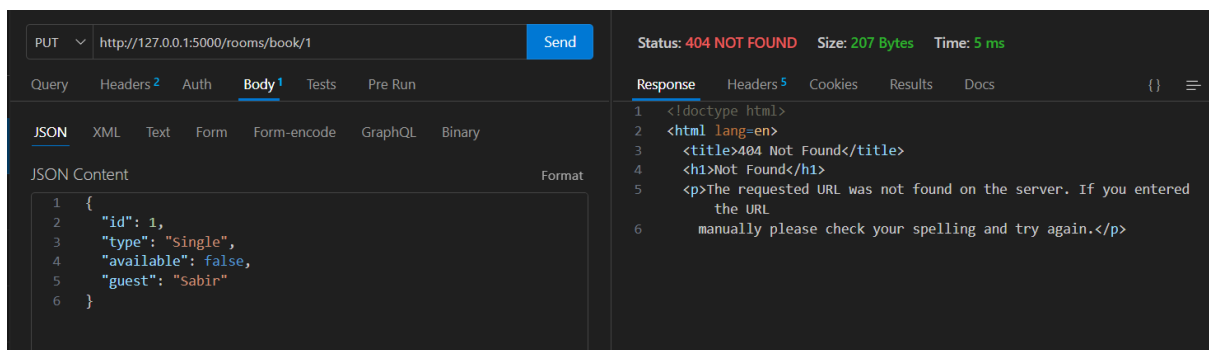
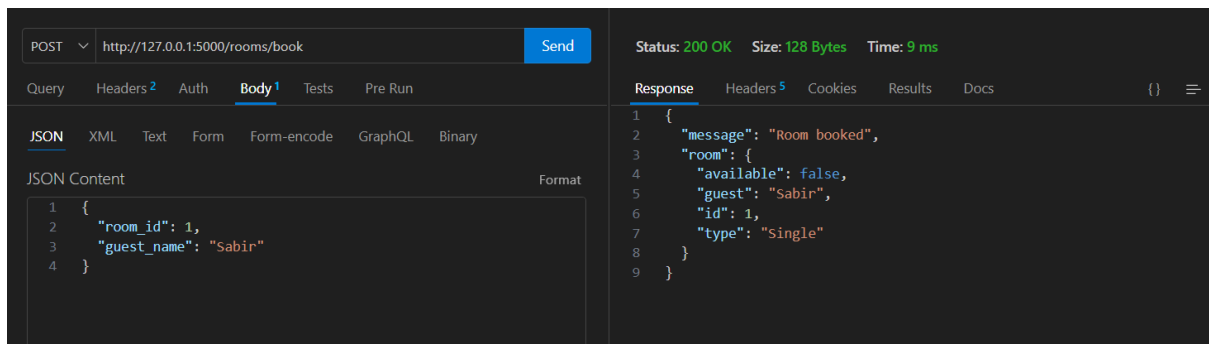
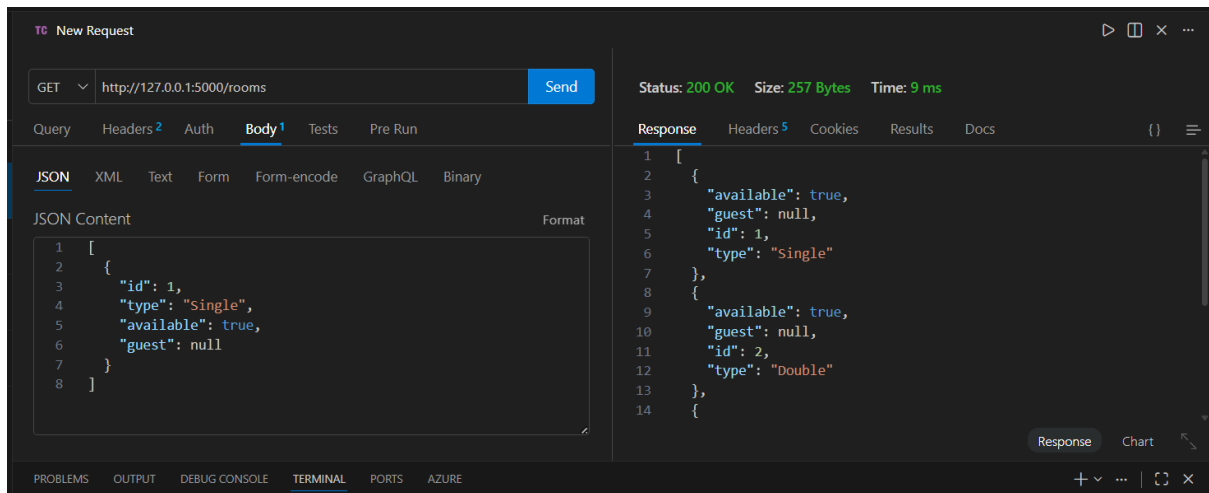
Code Screenshot :

```

174 from flask import Flask, request, jsonify
175 app = Flask(__name__)
176 # Sample room data (in-memory)
177 rooms = [
178     {"id": 1, "type": "Single", "available": True, "guest": None},
179     {"id": 2, "type": "Double", "available": True, "guest": None},
180     {"id": 3, "type": "Suite", "available": True, "guest": None}
181 ]
182 # -----
183 # GET /rooms → Get available rooms
184 # -----
185 @app.route('/rooms', methods=['GET'])
186 def get_rooms():
187     available_rooms = [room for room in rooms if room["available"]]
188     return jsonify(available_rooms), 200
189 # -----
190 # POST /rooms/book → Book a room
191 # -----
192 @app.route('/rooms/book', methods=['POST'])
193 def book_room():
194     data = request.get_json()
195     if not data or 'room_id' not in data or 'guest_name' not in data:
196         return jsonify({"error": "room_id and guest_name required"}), 400
197     for room in rooms:
198         if room["id"] == data["room_id"]:
199             if not room["available"]:
200                 return jsonify({"error": "Room already booked"}), 400
201             room["available"] = False
202             room["guest"] = data["guest_name"]
203             return jsonify({"message": "Room booked", "room": room}), 200
204     return jsonify({"error": "Room not found"}), 404
205 # -----
206 # GET /rooms/<id> → Booking details
207 # -----
208 @app.route('/rooms/<int:id>', methods=['GET'])
209 def get_room(id):
210     for room in rooms:
211         if room["id"] == id:
212             return jsonify(room), 200
213     return jsonify({"error": "Room not found"}), 404
214 # -----
215 # DELETE /rooms/<id> → Cancel booking
216 # -----
217 @app.route('/rooms/<int:id>', methods=['DELETE'])
218 def cancel_booking(id):
219     for room in rooms:
220         if room["id"] == id:
221             if room["available"]:
222                 return jsonify({"error": "Room is not booked"}), 400
223             room["available"] = True
224             room["guest"] = None
225             return jsonify({"message": "Booking cancelled"}), 200
226     return jsonify({"error": "Room not found"}), 404
227 # -----
228 # Run server
229 # -----
230 if __name__ == '__main__':
231     app.run(debug=True)

```

Output Screenshot:



Explanation/Justification/Observation (100 words / 5 – 6 sentence) :

The hotel room booking API effectively demonstrates RESTful principles by managing room availability and bookings through well-defined endpoints. It uses an in-memory data structure to store room details, making it suitable for testing and learning purposes. The API ensures proper validation by checking for missing inputs and preventing booking of already reserved rooms, enhancing reliability. Error handling is implemented to return meaningful messages for invalid operations such as booking unavailable rooms or cancelling non-existent bookings. However, data persistence is not maintained across server restarts. Overall, the API is simple, structured, and provides a solid foundation for building real-world booking systems.

Task-04

Final Optimal Prompt:

Create a RESTful API for an online voting system using any backend framework (e.g., Flask, Node.js). Implement the following endpoints:

- GET /candidates → Retrieve list of candidates
- POST /vote → Cast a vote
- GET /results → View voting results

Ensure all data is handled in JSON format and implement in-memory validation to allow only one vote per user.

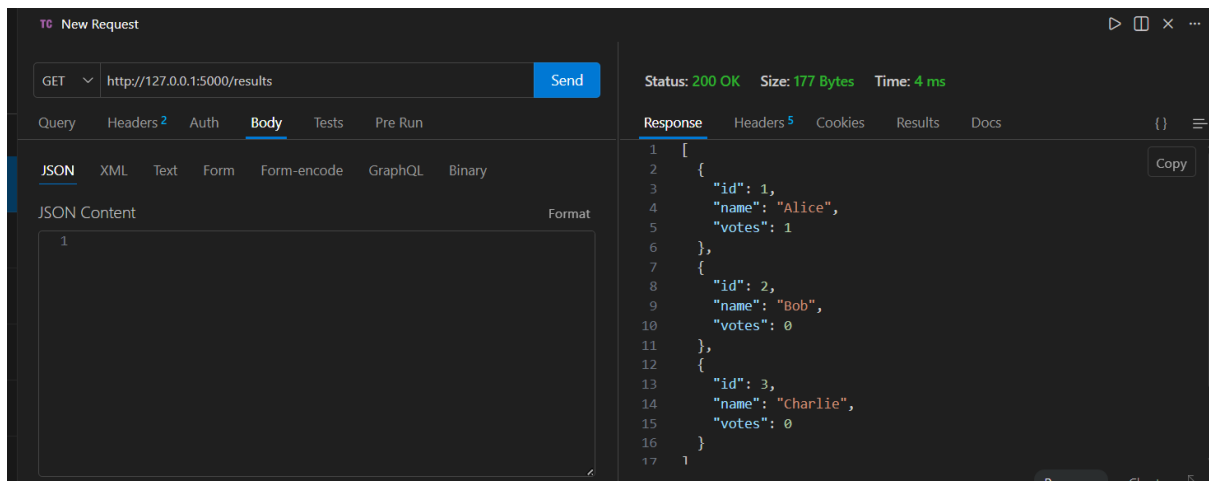
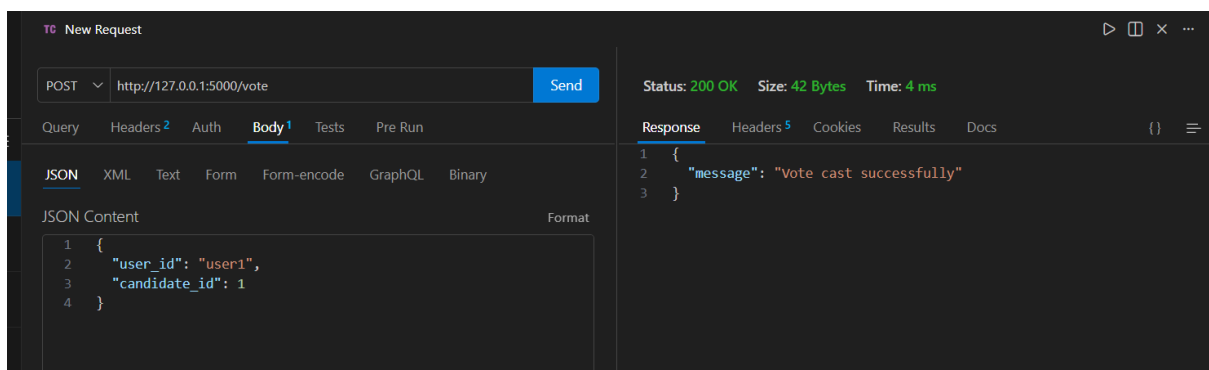
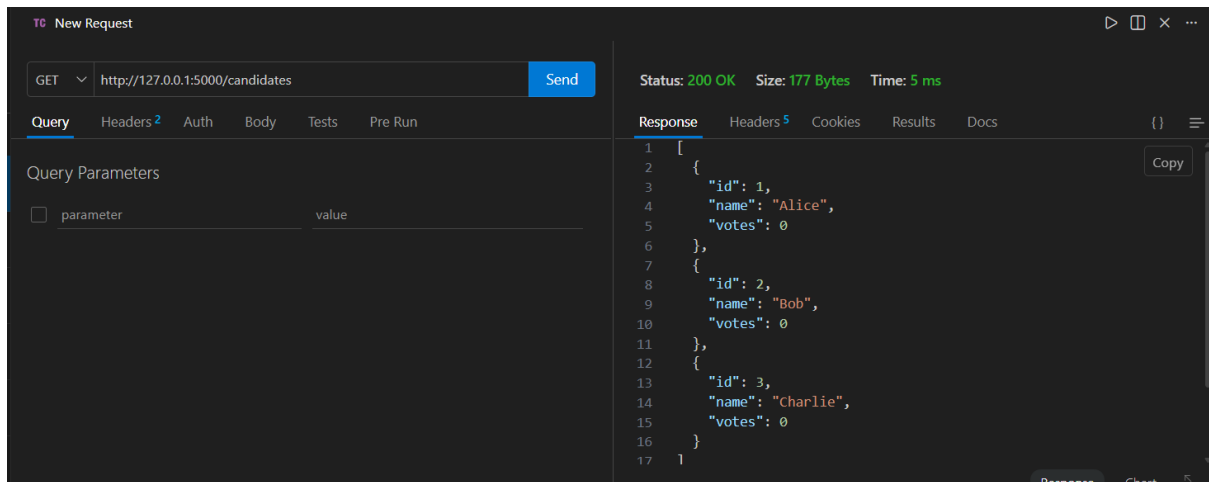
Code Screenshot :

```

243 from flask import Flask, request, jsonify
244 app = Flask(__name__)
245 # In-memory data
246 candidates = [
247     {"id": 1, "name": "Alice", "votes": 0},
248     {"id": 2, "name": "Bob", "votes": 0},
249     {"id": 3, "name": "Charlie", "votes": 0}
250 ]
251 # Track users who have voted
252 voted_users = set()
253 # -----
254 # GET /candidates → List candidates
255 # -----
256 @app.route('/candidates', methods=['GET'])
257 def get_candidates():
258     return jsonify(candidates), 200
259 # -----
260 # POST /vote → Cast vote
261 # -----
262 @app.route('/vote', methods=['POST'])
263 def vote():
264     data = request.get_json()
265     # Validation
266     if not data or 'user_id' not in data or 'candidate_id' not in data:
267         return jsonify({"error": "user_id and candidate_id required"}), 400
268     user_id = data["user_id"]
269     candidate_id = data["candidate_id"]
270     # Check if user already voted
271     if user_id in voted_users:
272         return jsonify({"error": "User has already voted"}), 400
273     # Find candidate
274     for candidate in candidates:
275         if candidate["id"] == candidate_id:
276             candidate["votes"] += 1
277             voted_users.add(user_id)
278             return jsonify({"message": "Vote cast successfully"}), 200
279     return jsonify({"error": "Candidate not found"}), 404
280 # -----
281 # GET /results → Voting results
282 # -----
283 @app.route('/results', methods=['GET'])
284 def get_results():
285     return jsonify(candidates), 200
286 # -----
287 # Run server
288 # -----
289 if __name__ == '__main__':
290     app.run(debug=True)

```

Output Screenshot:



Explanation/Justification/Observation (100 words / 5 – 6 sentence) :

The online voting system API effectively demonstrates RESTful principles by providing endpoints to retrieve candidates, cast votes, and view results. It uses an in-memory data structure to store candidates and vote counts, making it simple and suitable for testing or educational purposes. The implementation includes validation to ensure that each user can vote only once, enhancing fairness and preventing duplicate voting. Error handling is incorporated to manage invalid inputs and non-existent candidates. However, since the data

is not persistent, all information is lost upon server restart. Overall, the API is well-structured, efficient, and forms a strong base for real-world voting applications

Tack-05

Final Optimal Prompt:

Create a RESTful API for managing teacher records using any backend framework (e.g., Flask, Node.js). Implement the following endpoints:

- GET /teachers → Retrieve all teachers
- POST /teachers → Add a new teacher
- PUT /teachers/{id} → Update teacher details
- DELETE /teachers/{id} → Remove a teacher

Ensure all requests and responses use JSON format and include proper error handling for invalid or missing data.

Code Screenshot :

```

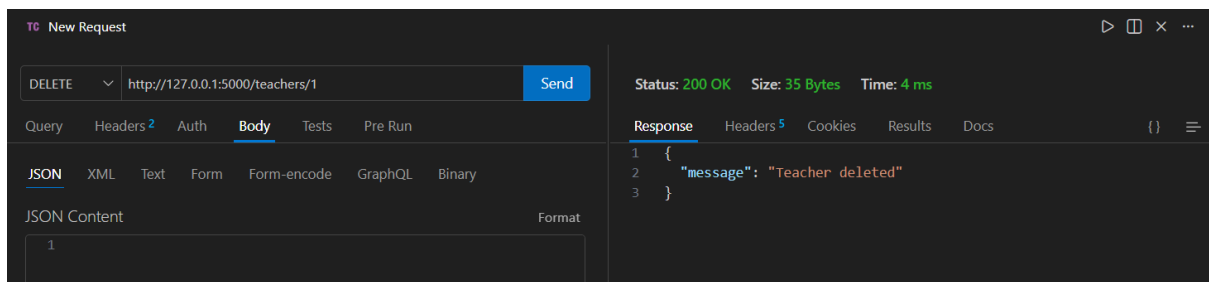
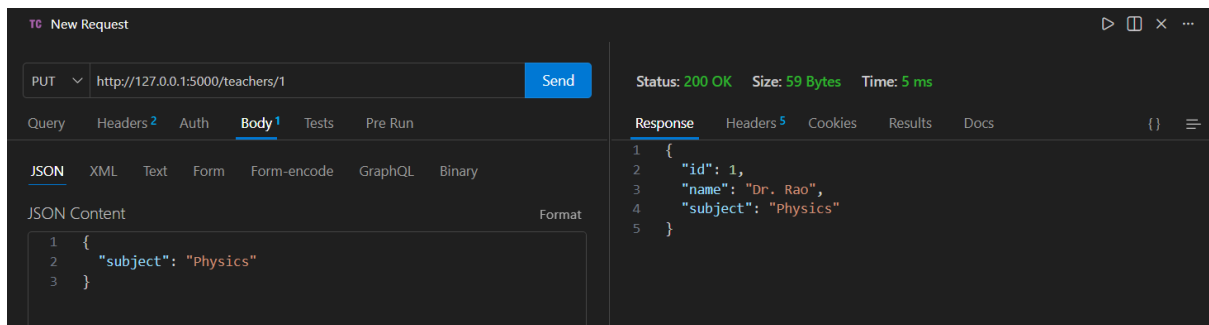
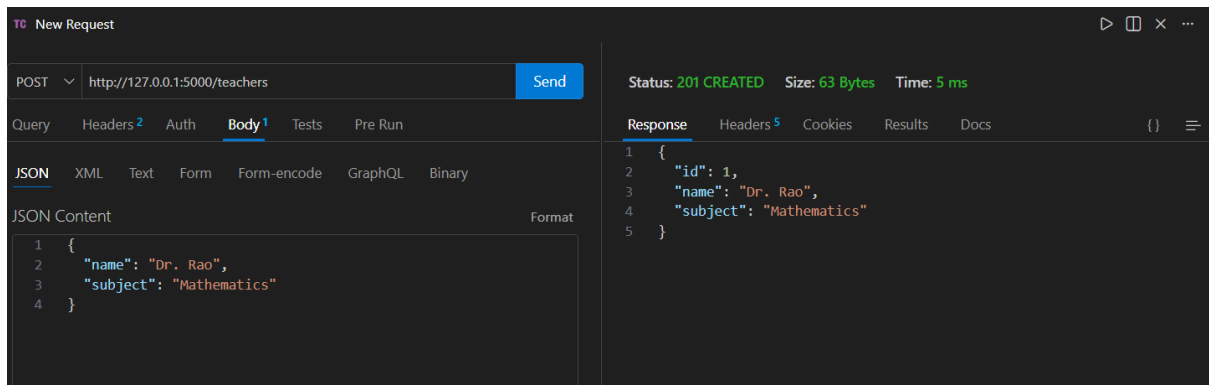
302 '''
303 from flask import Flask, request, jsonify
304 app = Flask(__name__)
305 # In-memory database
306 teachers = []
307 current_id = 1
308 # -----
309 # GET /teachers → Get all teachers
310 # -----
311 @app.route('/teachers', methods=['GET'])
312 def get_teachers():
313     return jsonify(teachers), 200
314 # -----
315 # POST /teachers → Add teacher
316 # -----
317 @app.route('/teachers', methods=['POST'])
318 def add_teacher():
319     global current_id
320     data = request.get_json()
321     # Validation
322     if not data or 'name' not in data or 'subject' not in data:
323         return jsonify({"error": "Name and subject are required"}), 400
324     teacher = {
325         "id": current_id,
326         "name": data["name"],
327         "subject": data["subject"]
328     }
329     teachers.append(teacher)
330     current_id += 1
331     return jsonify(teacher), 201
332 # -----
333 # PUT /teachers/<id> → Update teacher
334 # -----
335 @app.route('/teachers/<int:id>', methods=['PUT'])
336 def update_teacher(id):
337     data = request.get_json()
338     if not data:
339         return jsonify({"error": "No data provided"}), 400
340     for teacher in teachers:
341         if teacher["id"] == id:
342             teacher["name"] = data.get("name", teacher["name"])
343             teacher["subject"] = data.get("subject", teacher["subject"])
344             return jsonify(teacher), 200
345     return jsonify({"error": "Teacher not found"}), 404
346 # -----
347 # DELETE /teachers/<id> → Remove teacher
348 # -----
349 @app.route('/teachers/<int:id>', methods=['DELETE'])
350 def delete_teacher(id):
351     global teachers
352     for teacher in teachers:
353         if teacher["id"] == id:
354             teachers = [t for t in teachers if t["id"] != id]
355             return jsonify({"message": "Teacher deleted"}), 200
356     return jsonify({"error": "Teacher not found"}), 404
357 # -----
358 # Run server
359 # -----
360 if __name__ == '__main__':
361     app.run(debug=True)

```

Output Screenshot:

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** http://127.0.0.1:5000/teachers
- Status:** 200 OK
- Size:** 3 Bytes
- Time:** 5 ms
- Response Body:** []



Explanation/Justification/Observation (100 words / 5 – 6 sentence) :

The teacher management API demonstrates a clear implementation of RESTful principles by providing endpoints for creating, retrieving, updating, and deleting teacher records. It uses an in-memory data structure, making it suitable for testing and small-scale applications. Proper validation ensures that required fields such as name and subject are provided, and error handling improves reliability by returning meaningful responses for invalid operations. Each teacher is uniquely identified by an ID, enabling efficient updates and deletions. However, data persistence is not maintained across server restarts. Overall, the API is simple, well-structured, and serves as a strong foundation for building scalable educational management systems.